ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

BLM4538 - IOS İLE MOBİL UYGULAMA GELİŞTİRME II

CHECKPOINT

Game Tracking and Social Mobile Application

İsmail Eren Özçınar - 22290161

GitHub Linki

https://github.com/ismailerenozcinar

Video Oynatma Listesi Linki

https://www.youtube.com/playlist?list=PLZ4F78rKHrCt4r3aYinL0fj9jqY4HyAs6

1 Giriş

Checkpoint, kullanıcıların oynadıkları video oyunlarını takip edebilecekleri, yeni oyunlar keşfedebilecekleri ve oyunlara inceleme yazabilecekleri bir mobil uygulamadır.

Uygulama sayesinde kullanıcılar kendi oyun kütüphanelerini oluşturabilir, oynadıkları oyunları kategorilere ayırabilir ve oyunlara puan vererek diğer kullanıcılarla etkileşim kurabilirler.

Uygulama üç ana bileşenden oluşmaktadır:

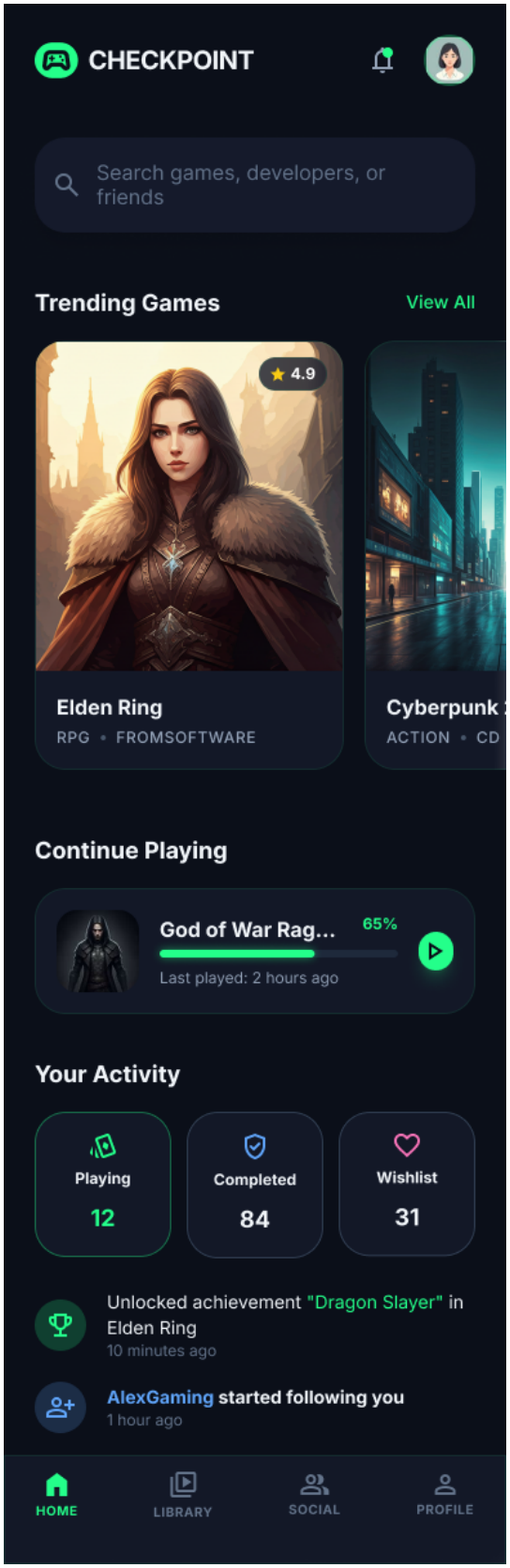
- Frontend: React Native kullanılarak geliştirilen mobil uygulama
- Backend: C# ile geliştirilen ASP.NET Core Web API
- Database: Microsoft SQL Server ilişkisel veritabanı

Ayrıca oyun bilgilerini almak için harici bir oyun veri API'si kullanılacaktır.

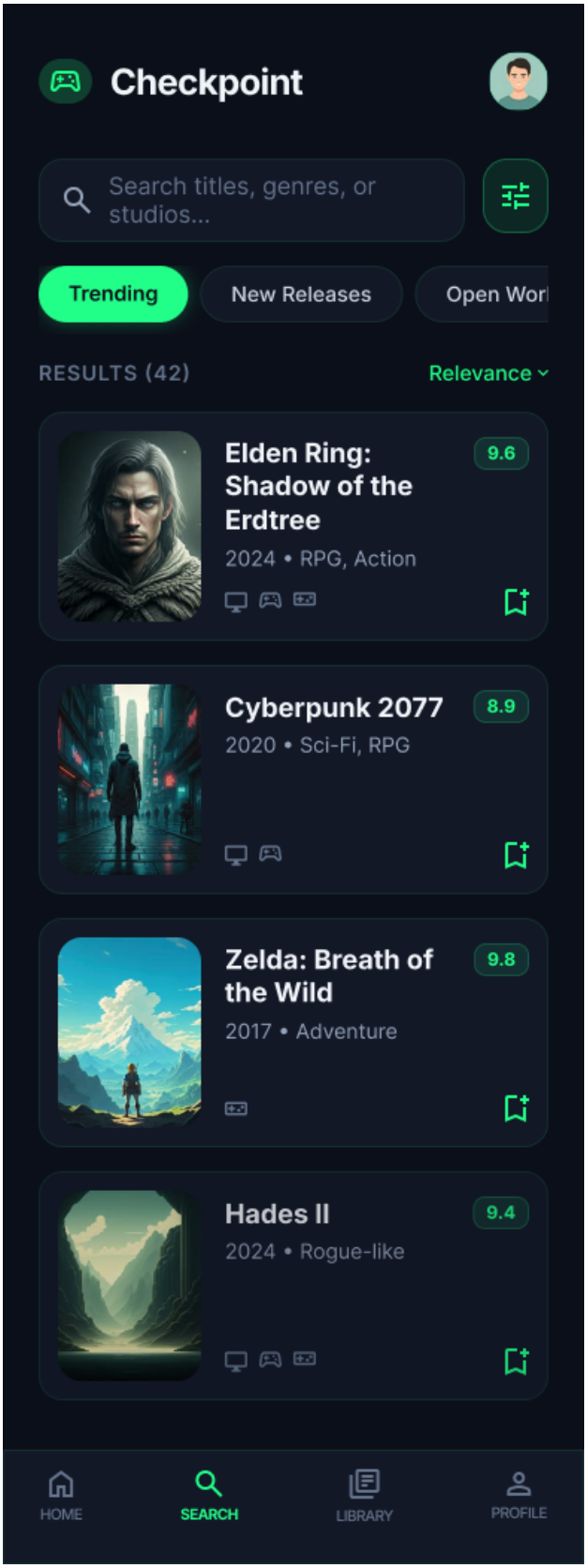
2 Mobil Arayüz Tasarımları

Uygulamanın kullanıcı arayüzü geliştirme sürecinden önce Figma platformu kullanılarak tasarlanacaktır. Tasarlanan ekranlar kullanıcı deneyimini planlamak ve uygulamanın genel görünümünü belirlemek amacıyla hazırlanacaktır.

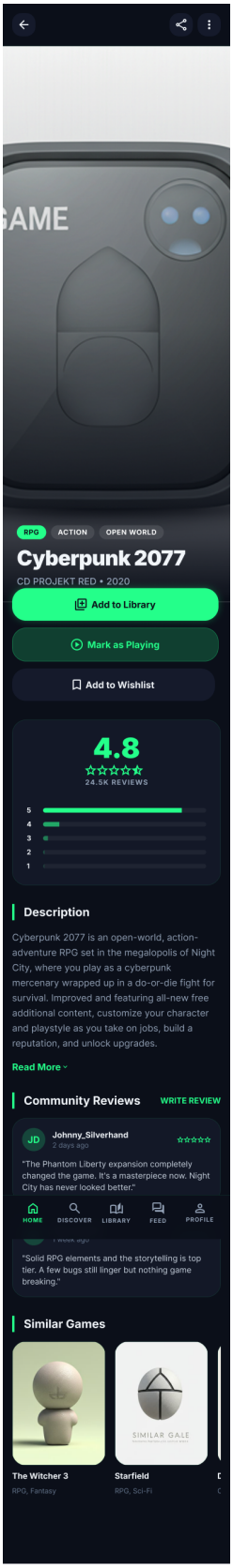
2.1 Ana Sayfa (Home Screen)



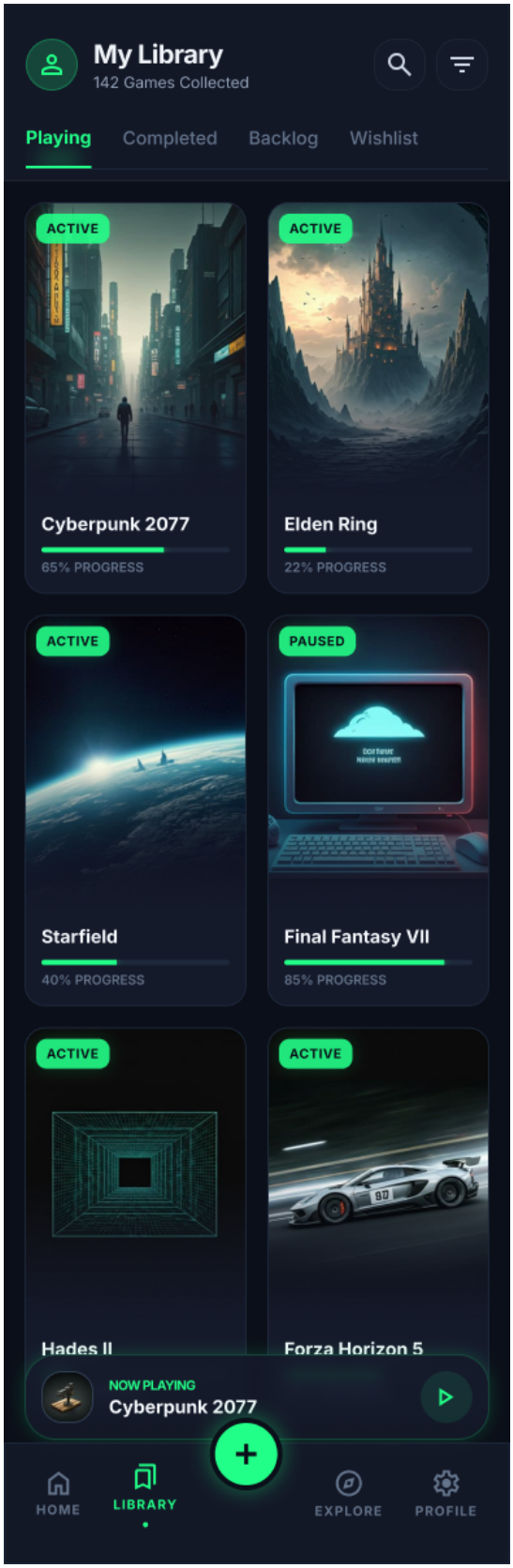
2.2 Oyun Arama Ekranı (Search Screen)



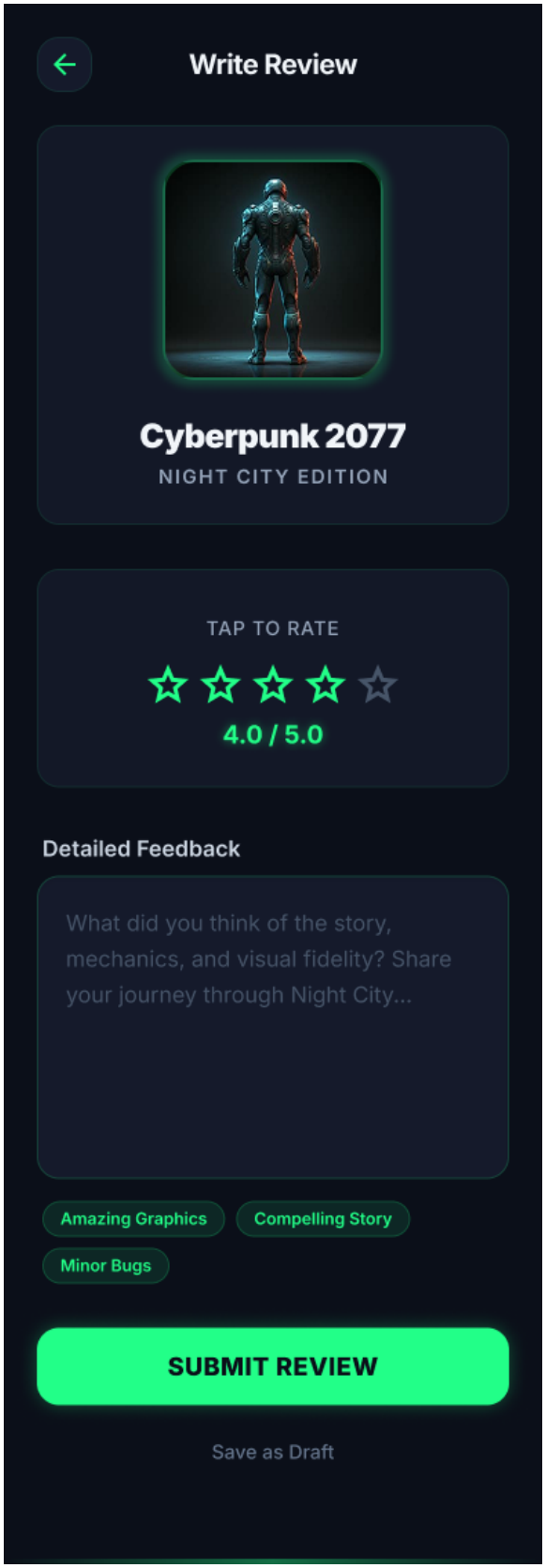
2.3 Oyun Detay Ekranı



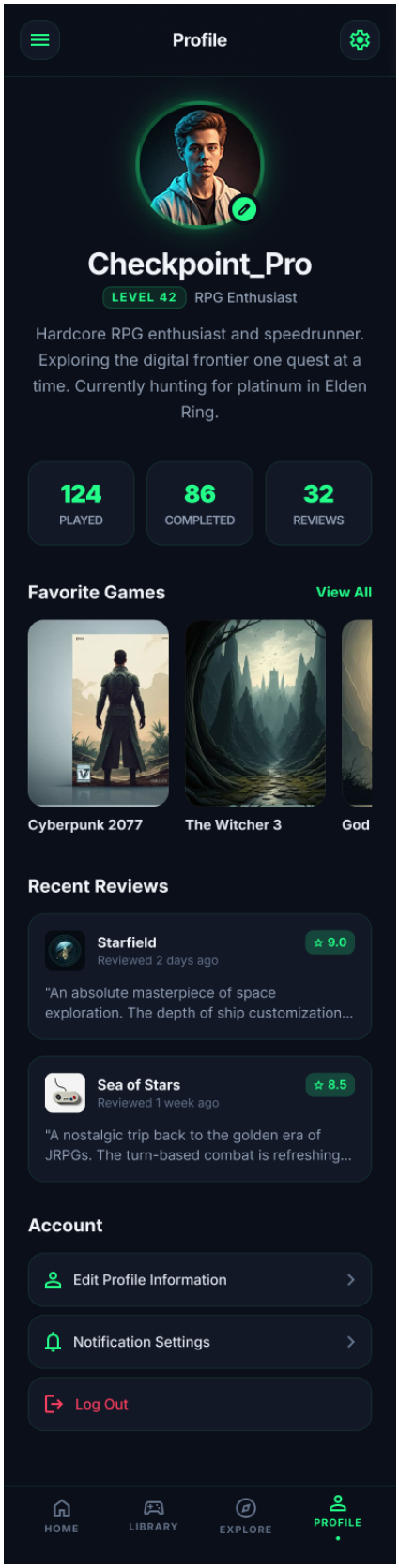
2.4 Oyun Kütüphanesi (Library Screen)



2.5 İnceleme Yazma Ekranı



2.6 Profil Ekranı



3 1. Hafta: Proje İskeleti ve Mimari Planlama

İlk hafta kapsamında uygulamanın temel iskeleti oluşturulmuş ve kullanılacak mimariler kod bazında kurgulanmıştır:

- **Tasarım:** Figma üzerinden uygulamanın ana ekran tasarımları (Koyu tema ve yeşil accent kullanılarak) tamamlandı.
- **Frontend İskeleti:** Expo ile React Native projesi oluşturuldu. Uygulama içinde Bottom Tab ve Stack navigasyon yapıları kuruldu. Temel renk ve tema sabitleri `theme.js` dosyasına çıkarıldı.
- **Backend İskeleti:** ASP.NET Core Web API projesi, 4 katmanlı (API, Business, DataAccess, Entities) SOLID mimariye uygun olarak oluşturuldu.
- **Veritabanı Tasarımı:** MSSQL kullanılmak üzere 8 adet Entity modeli (User, Game, Review, UserLibrary, vs.) ve DbContext (Entity Framework Core) yapılandırıldı. Tablolar arası ilişkiler (One-to-Many, Many-to-Many) kuruldu.

4 2. Hafta: Kullanıcı Kimlik Doğrulama (Authentication)

İkinci hafta geliştirmeleri kapsamında sistemin en kritik modüllerinden olan "Kullanıcı Kayıt ve Giriş" özellikleri baştan sona (Frontend & Backend) uygulanmıştır:

- **Backend Düzeyinde Güvenlik:** C# katmanına `JWT` tabanlı kimlik doğrulama mekanizması eklendi. Kullanıcı kayıt olduğunda parolalarının düz metin olarak saklanmasını engellemek amacıyla `BCrypt.Net` algoritması ile "hash"lenme sağlandı.
- **Servis Katmanı (Business):** `IAuthService` ve `AuthService` aracılığıyla *Login* ve *Register* iş mantıkları yazıldı. DataAccess katmanında bulunan Data Access Layer (DAL) sınıfları ile veritabanı sorguları yapıldı.
- **React Native Arayüzleri:** Kimlik doğrulama akışı için `LoginScreen` ve `RegisterScreen` sayfaları tasarlandı.
- **Oturum Yönetimi (AsyncStorage):** Kullanıcı başarılı giriş yaptığında cihazın yerel önbellek birimine (`AsyncStorage`) elde edilen Token kaydedildi. `App.js` dosyasındaki

akış kontrolü ile tokeni olan kullanıcının uygulamayı açtığında direkt ana sayfaya gitmesi (Auth Flow) engelsiz bir şekilde koordine edildi.

- **Axios Interceptor:** API ile iletişim kuran `api.js` doyası güncellenerek giden her arka plan isteğinin "Header" (Başlık) kısmına aktif JWT Token'ın otomatik yerleştirilmesi (`Bearer <token>`) eklendi.
- **Veritabanının Ayağa Kaldırılması:** `dotnet ef migrations add InitialCreate` komutu ile veritabanı göçü oluşturuldu ve MS SQL Server üzerinde tablolar aktifleştirildi.

5 3. Hafta: Ana Sayfa ve Oyun Listesi

Üçüncü hafta kapsamında backend'de oyun veri katmanı sıfırdan inşa edilmiş, veritabanına başlangıç oyun verisi eklenmiş ve mobil ana sayfa gerçek API verileriyle entegre edilmiştir.

Backend Geliştirmeleri

- **Veri Erişim Katmanı (DataAccess):** Oyun sorgularını yönetmek için `IGameDal` arayüzü ve `EfGameDal` sınıfı oluşturuldu. `GetTrendingAsync`, `GetByIdWithGenresAsync`, `SearchAsync` ve `GetSimilarAsync` metotları Entity Framework Core'un `Include().ThenInclude()` zinciri aracılığıyla tür (genre) bilgilerini birleştirerek (JOIN) çekecek biçimde yazıldı.
- **İş Katmanı (Business):** `IGameService` arayüzünü karşılayan `GameService` sınıfı oluşturuldu. `GamesController`, artık doğrudan veritabanı yerine bu servis üzerinden çalışmaktadır.
- **Seed Verisi:** `CheckpointDbContext` içine *Elden Ring, Cyberpunk 2077, God of War Ragnarök, Hades, The Witcher 3* dahil 10 oyun ve bu oyunların tür ilişkilerini (toplamda 29 kayıt) kapsayan seed verisi eklendi. `dotnet ef migrations add Week3_GameSeedData` ve `dotnet ef database update` komutlarıyla veritabanı güncellendi.
- **Controller Projeksiyonu:** `GamesController`'daki endpoint'ler anonim DTO projeksiyonuyla döngüsel referans (circular reference) sorununu engelleyecek biçimde güncellendi.

- **Bağımlılık Enjeksiyonu:** `Program.cs` dosyasına `IGameDal / EfGameDal` ve `IGameService / GameService` çiftleri `AddScoped` ile kaydedildi.

Frontend Geliştirmeleri

- **API Yapılandırma Dosyası:** `src/config/api.config.js` adlı yeni bir dosya oluşturuldu. Geliştirici, test ortamına göre (web tarayıcısı, fiziksel cihaz veya emülatör) tek satırı değiştirerek API host adresini kolayca güncelleyebilmektedir. `api.js` içindeki sabit `localhost` adresi bu konfigürasyona bağlandı.

- **HomeScreen API Entegrasyonu:** Ana sayfa ekranı `gameService.getTrending()` ve `libraryService.getMyLibrary()` çağrılarıyla gerçek verilerle beslenir hale getirildi. Bileşen; yükleniyor (loading spinner), hata mesajı ve *pull-to-refresh* durumlarını yönetmektedir. Statik örnek veri tamamen kaldırıldı.

- **QR Kod / Ağ Sorunu Çözümü:** Bilgisayarın Ethernet adaptörüne bağlı olması, telefonun ise aynı modeme Wi-Fi üzerinden bağlanması nedeniyle Expo Go QR kodu okutulduğunda bağlantı sağlanamıyordu. `package.json` dosyasına `"tunnel": "expo start --tunnel"` scripti eklenerek ngrok tüneli üzerinden cihaz bağımsız bağlantı sağlandı.

- **Expo SDK Güncellemesi:** Telefonda yüklü Expo Go'nun SDK 54 gerektirmesi nedeniyle proje SDK 52'den SDK 54'e yükseltildi; `react-native` 0.76 $\rightarrow$ 0.81, `react` 18 $\rightarrow$ 19 olarak güncellendi.

6 4. Hafta: Oyun Detay Ekranı ve Arama

Dördüncü hafta geliştirmeleri kapsamında *GameDetailScreen* gerçek API verileriyle tam olarak hayata geçirilmiş, *SearchScreen*'e canlı arama, tür filtreleme ve sıralama özellikleri eklenmiştir.

GameDetailScreen — Tam Uygulama ve Animasyonlar

- **API Entegrasyonu:** Ekran açıldığında `GET /api/games/{id}`, `GET /api/games/{id}/similar` ve `GET /api/reviews/game/{id}` uç noktaları `Promise.allSettled` ile paralel olarak

çağrılmaktadır. Herhangi birinin başarısız olması diğerlerinin gösterimini engellememektedir.

- **Parallax Cover Animasyonu:** `Animated.ScrollView` ile elde edilen kaydırma değeri (`scrollY`), kapak görselinin dikey konumuna ve ölçeğine bağlanmıştır. Kullanıcı sayfayı aşağı kaydırdıkça kapak resmi yarı hızda yukarı kayar; yukarı çekildiğinde ise görüntü hafifçe büyüyerek dolgu efekti oluşturur. Tüm animasyonlar `useNativeDriver: true` ile JS iş parçacığı dışında çalışmaktadır.
- **Sticky Header Animasyonu:** Belirli bir kaydırma mesafesine ulaşıldığında oyun adını içeren yarı-saydam bir üst başlık `opacity` interpolasyonuyla belirmektedir. Kullanıcı yukarı döndüğünde başlık yeniden kaybolmaktadır.
- **Açıklama Alanı (Expand/Collapse):** Oyun açıklaması varsayılan olarak 160 karakterde kırpılmakta, "*Read More* " butonuyla tam metin görüntülenebilmektedir.
- **Benzer Oyunlar:** Yatay kaydırmalı listenin her öğesine tıklandığında `navigation.push` ile yeni bir *GameDetailScreen* açılmaktadır; bu sayede "derinleşme" akışı sağlanmıştır.

SearchScreen — Canlı Arama, Filtreleme ve Sıralama

- **Debounce ile Canlı Arama:** Kullanıcı arama kutusuna yazdıkça 350 ms'lik bir bekleme süresi (debounce) uygulanmakta ve ardından `GET /api/games/search?query=&genre=` uç noktasına istek gönderilmektedir. Bu yaklaşım gereksiz ağ çağrılarını engeller.
- **Tür (Genre) Filtre Chip'leri:** Yatay kaydırmalı "All, RPG, Action, Adventure, Open World, Sci-Fi, Fantasy, Indie, Rogue-like, Strategy" chip'leri yer almaktadır. Seçim değiştiğinde arama, seçili tür parametresiyle yeniden tetiklenir.
- **Sıralama Modal'ı:** Ekranın altından sürgü efektiyle açılan bir modal üç sıralama seçeneği sunmaktadır: *En Yüksek Puan* (API varsayılanı), *En Yeni* (yayın yılına göre, istemci tarafı), *İsim A–Z* (alfabetik, istemci tarafı).
- **Navigasyon:** Listeden herhangi bir oyuna tıklandığında `SearchStack` içindeki *GameDetailScreen*'e geçilmektedir. Bu geçişin çalışabilmesi için `SearchStackNavigator` oluşturulmuş ve `AppNavigator`'a eklenmiştir.

Diğer İyileştirmeler

- **Web Uyumluluğu (Alert Sorunu):** `LoginScreen` ve `RegisterScreen`'de React Native'in `Alert.alert()` API'si tarayıcıda sessizce yok sayıldığından doğrulama ve hata mesajları çalışmıyordu. Her iki ekranda da `Alert` kullanımı kaldırılarak ekran içi kırmızı hata kutusu bileşenleriyle değiştirildi. Ayrıca *Login* ekranından *Register* ekranına geçişte `onLogin` fonksiyonu açıkça parametre olarak iletilecek biçimde düzeltildi.